GUVI × HCL · DEEP LEARNING CAPSTONE

# RecycleVision

Garbage image classification with deep learning: 10 waste categories, transfer learning and a Streamlit app with Grad-CAM explanations.

| 96.5% | 0.963 | 16,153 | 18 MB |
|---|---|---|---|
| test accuracy (EfficientNetB0) | test macro-F1 | unique images, 10 classes | deployed model size |

**Author** Aryaman Bhatnagar

**Date** 20 October 2026

**Code** https://github.com/aryaman2213/RecycleVision-Garbage-Image-Classification-Using-Deep-Learning

**App** run locally: `streamlit run app/streamlit_app.py`

## 1. Problem statement and business use

Build a deep learning model that classifies an image of waste into its material category, and deploy it behind a simple interface, so recycling can be sorted automatically from a photo.

- **Smart recycling bins:** a camera-equipped bin routes each item to the correct compartment.
- **Municipal waste management:** pre-sorting on conveyor lines cuts manual labour and contamination.
- **Education:** citizens photograph an item and learn which stream it belongs to; the app also gives disposal advice.
- **Environmental analytics:** counting predictions over time shows the waste composition and recycling trends.

## 2. Data: three datasets merged into one clean set

The brief offers three Kaggle datasets. Using all three gives more images and more variety, such as studio shots on white backgrounds and real-world web photos. However, **they overlap heavily**: most of the 10-class V2 dataset is copied from the 12-class one, and both contain the TrashNet photos. Concatenating them naively would put copies of the same photo in both training and test, inflating the test score. The pipeline therefore records the fate of every one of the 30,301 raw images in `data/manifest.csv`.

| Dataset | Raw images | Duplicates removed | Kept |
|---|---|---|---|
| Garbage Classification (12 classes) - mostafaabla | 15,515 | 2,460 | 10,187 |
| Garbage Classification V2 (10 classes) - sumn2u | 12,259 | 8,745 | 3,487 |
| Garbage Classification (6 classes, TrashNet) - asdasdasasdas | 2,527 | 27 | 2,479 |

| Step | Images | Removed |
|---|---|---|
| Raw images (3 zips) | 30,301 | |
| Unreadable / tiny / blank | | 7 |
| Exact + near duplicates (across sources) | | 11,232 |
| Same photo, conflicting labels | | 20 |
| Class cap (2,500) | | 2,889 |
| Final unique, labelled images | 16,153 | |

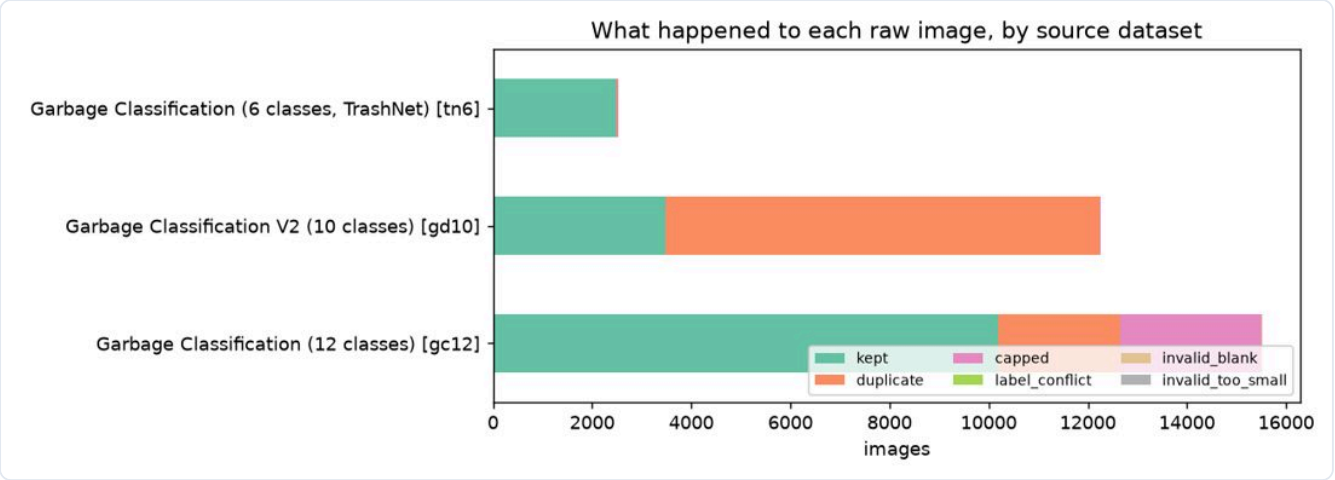
> **How duplicates were found.** Exact copies are matched by SHA-1 of the file bytes (11,232 duplicates in total, mostly byte-identical). Near-copies (resized or re-encoded) need a perceptual-hash distance ≤ 8 *and* a 32×32 thumbnail correlation ≥ 0.95. The second test was calibrated by inspecting borderline pairs: a pixel-difference test alone wrongly merged different bottles photographed on the same white background, and missed the same flyer re-shot with different lighting. Within each duplicate group, one copy is kept, preferring TrashNet and then the highest resolution.

**Taxonomy.** Brown, green and white glass were merged into **glass** (the brief asks for glass, not glass colour); "biological" is called **organic** as in the brief. The final classes are battery, cardboard, clothes, glass, metal,

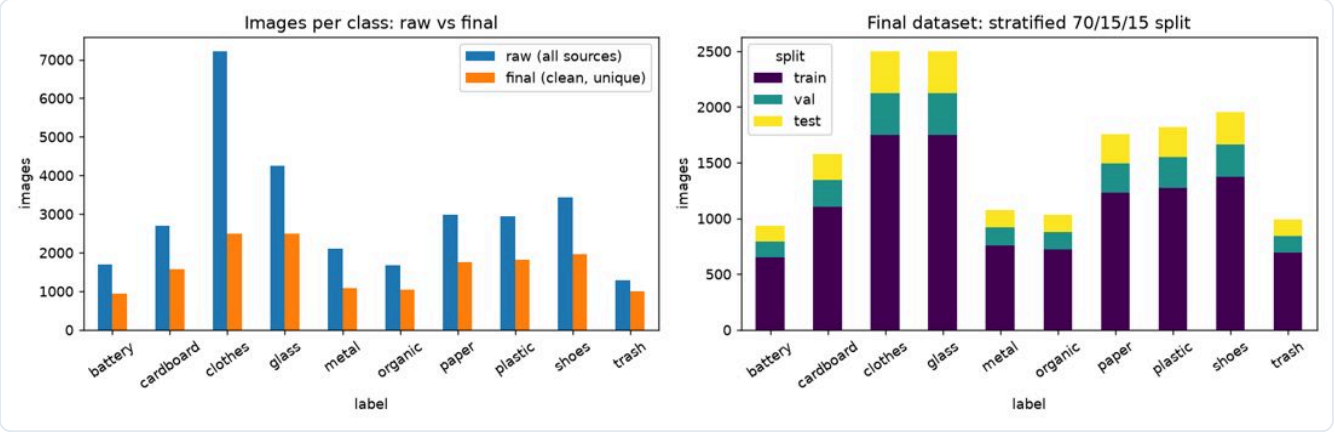
organic, paper, plastic, shoes, trash. **Class imbalance:** clothes was about a third of the raw data, so every class is capped at 2,500 images. The remaining imbalance (2.68× between the largest and smallest class) is handled with balanced class weights.

| Class | train | val | test | total |
|---|---|---|---|---|
| battery | 654 | 140 | 140 | 934 |
| cardboard | 1106 | 237 | 237 | 1580 |
| clothes | 1750 | 375 | 375 | 2500 |
| glass | 1750 | 375 | 375 | 2500 |
| metal | 755 | 162 | 162 | 1079 |
| organic | 725 | 155 | 156 | 1036 |
| paper | 1230 | 264 | 263 | 1757 |
| plastic | 1274 | 273 | 273 | 1820 |
| shoes | 1370 | 294 | 293 | 1957 |
| trash | 693 | 148 | 149 | 990 |

## 3. Exploratory data analysis




Fate of every raw image by source: most of the V2 dataset duplicates the 12-class set.

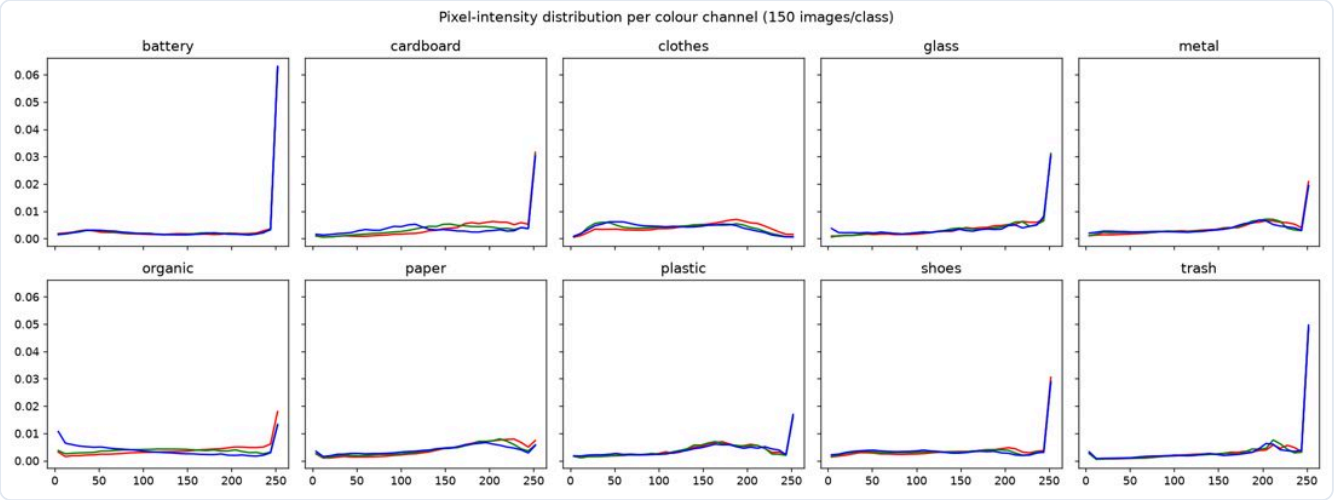



Images per class before and after cleaning, and the stratified 70/15/15 split.

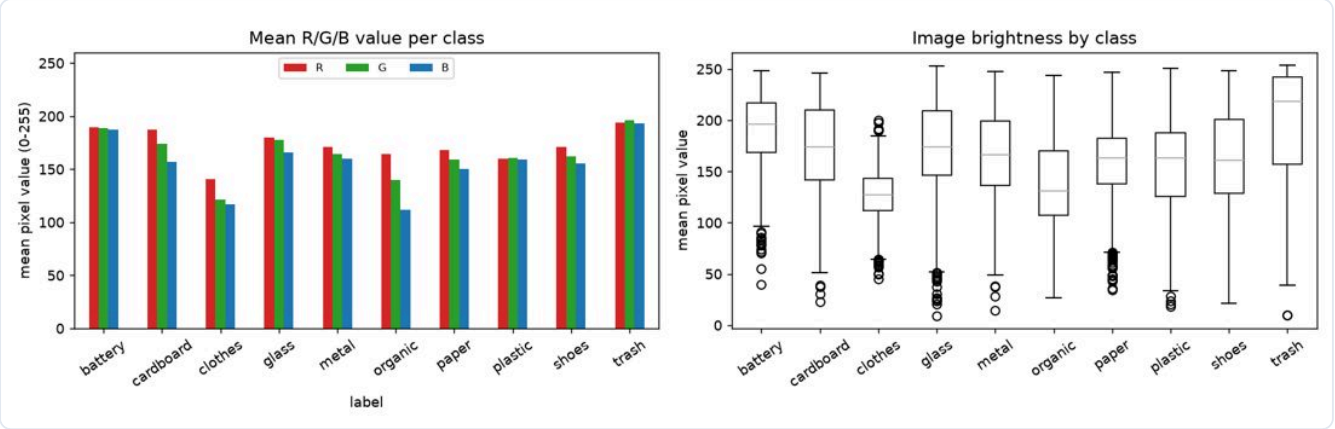
- **Classes:** after cleaning, the smallest classes are battery (934), trash (990), organic (1,036); the largest is capped at 2,500.
- **Image sizes** vary from 68 to 7786 px wide (median 328), so all images are resized to 224×224, the input size of the ImageNet backbones.
- **Colour:** trash and battery images are the brightest (mostly white studio backgrounds). Organic is the most red/yellow-shifted (blue channel far below red), and clothes are the darkest. Colour alone is therefore a strong but unreliable cue (background matters), which is a reason for colour and brightness augmentation.



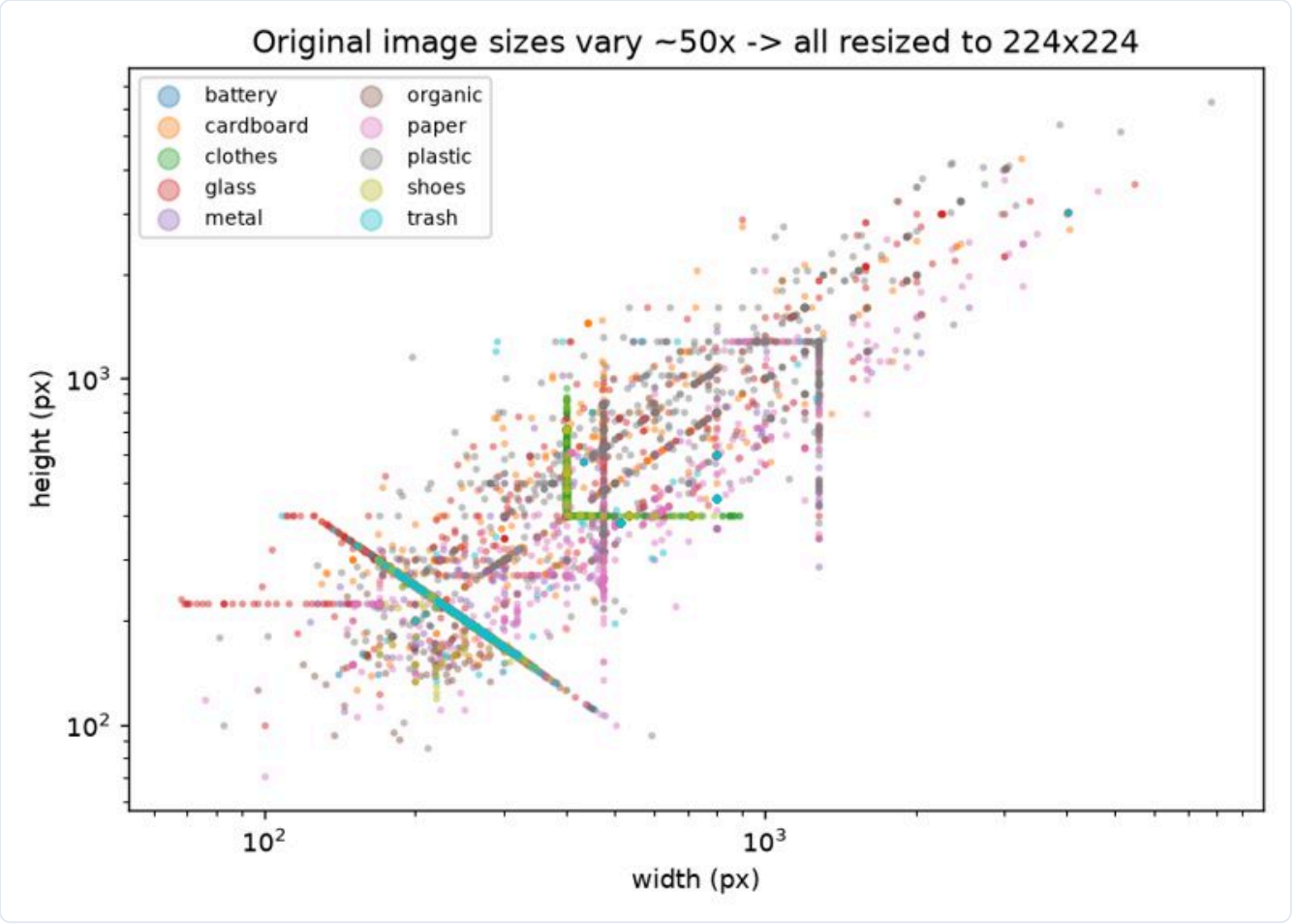

Random samples from each class after resizing.




Per-channel pixel-intensity distributions (150 images per class).




Mean R/G/B per class and brightness distribution.




Original image sizes (log-log) by class.

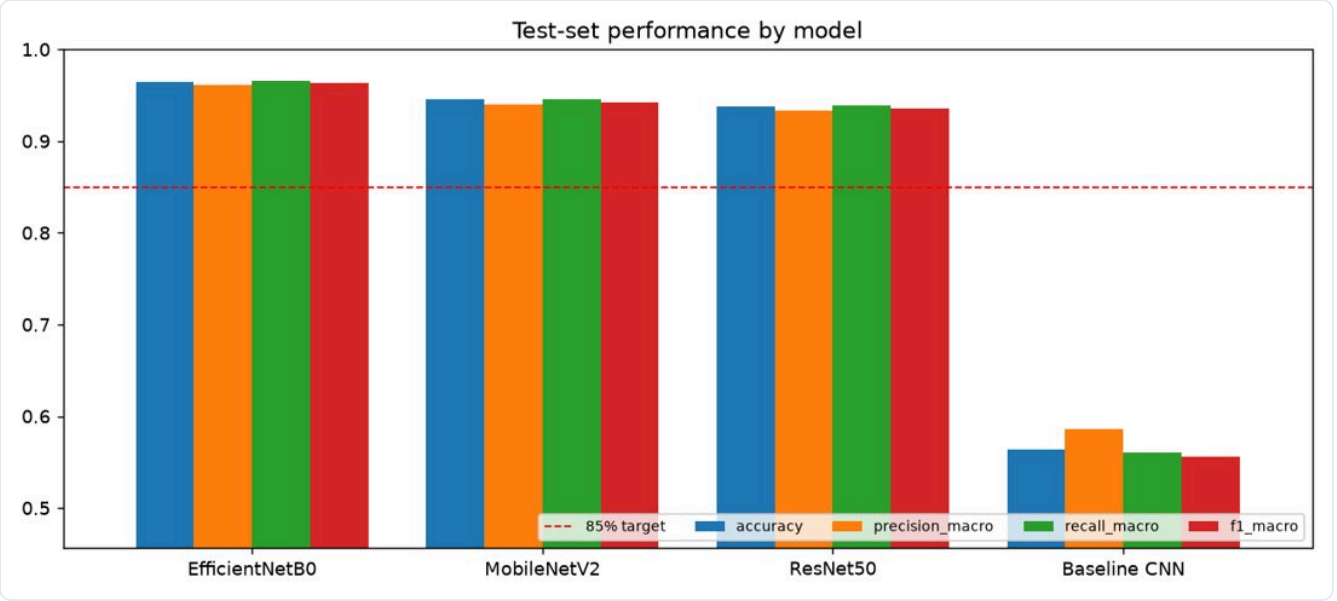
## 4. Preprocessing and models

- **Resize** to 224×224 and keep pixels at 0–255. **Normalisation is built into each model** (each backbone's own ImageNet scaling), so the app cannot apply the wrong normalisation.
- **Augmentation** (training only, inside the model): horizontal and vertical flips, rotation up to ±72°, zoom ±20%, shift ±10%, contrast and brightness ±20%. Waste can appear at any orientation and lighting.
- **Imbalance:** class cap plus balanced class weights; macro-F1 weights every class equally.
- **Baseline CNN** trained from scratch: 4 convolution blocks (32→256 filters, batch normalisation), global average pooling, then dense layers.
- **Transfer learning:** MobileNetV2, EfficientNetB0 and ResNet50 with ImageNet weights, plus a custom head (global average pooling → dropout → Dense 256 → dropout → softmax over 10 classes).
    - Phase 1 trains only the head (backbone frozen, AdamW, learning rate 1e-3).
    - Phase 2 unfreezes the top 35–45% of the backbone, with BatchNorm kept frozen, at learning rate 1e-4 with cosine decay.
    - Early stopping on validation loss restores the best epoch.
    - Trained with mixed precision on a Colab T4 GPU, then exported as float32 for CPU deployment.
- **Selection** is by validation macro-F1. The test set (2,423 images) is used only to report results.

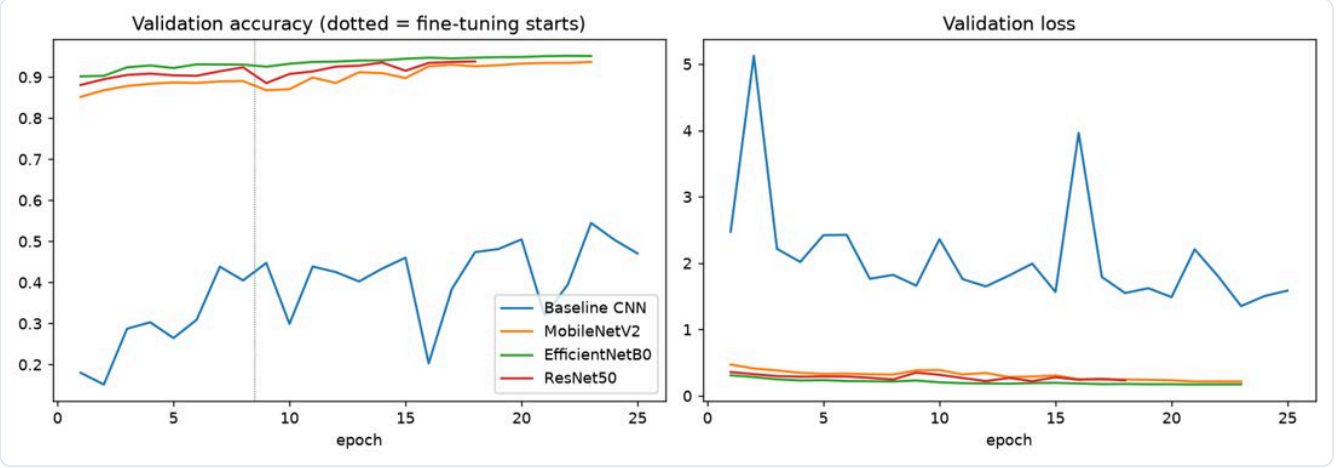
## 5. Results

| Model | Val macro-F1 | Test accuracy | Precision | Recall | Macro-F1 [95% CI] | Top-3 | FPR |
|---|---|---|---|---|---|---|---|
| **EfficientNetB0** | **0.948** | **96.5%** | **0.962** | **0.965** | **0.963 [0.955-0.971]** | **99.7%** | **0.40%** |
| MobileNetV2 | 0.933 | 94.5% | 0.940 | 0.945 | 0.943 [0.933-0.952] | 99.5% | 0.61% |
| ResNet50 | 0.931 | 93.8% | 0.934 | 0.939 | 0.936 [0.926-0.945] | 99.2% | 0.69% |
| Baseline CNN | 0.543 | 56.3% | 0.586 | 0.560 | 0.556 [0.534-0.575] | 82.9% | 4.89% |

**EfficientNetB0** has the highest validation macro-F1 and is the deployed model. On the held-out test set it reaches **96.5% accuracy** and **macro-F1 0.963**, with a 95% bootstrap interval of 0.955–0.971 and top-3 accuracy 99.7%. The brief's target of accuracy > 85% is **met**. Precision and recall are balanced (0.962 / 0.965), and the average false-positive rate per class is 0.40%. The from-scratch baseline shows how much ImageNet pre-training contributes.




Test accuracy, macro precision, recall and F1 per model; dashed line = 85% target.

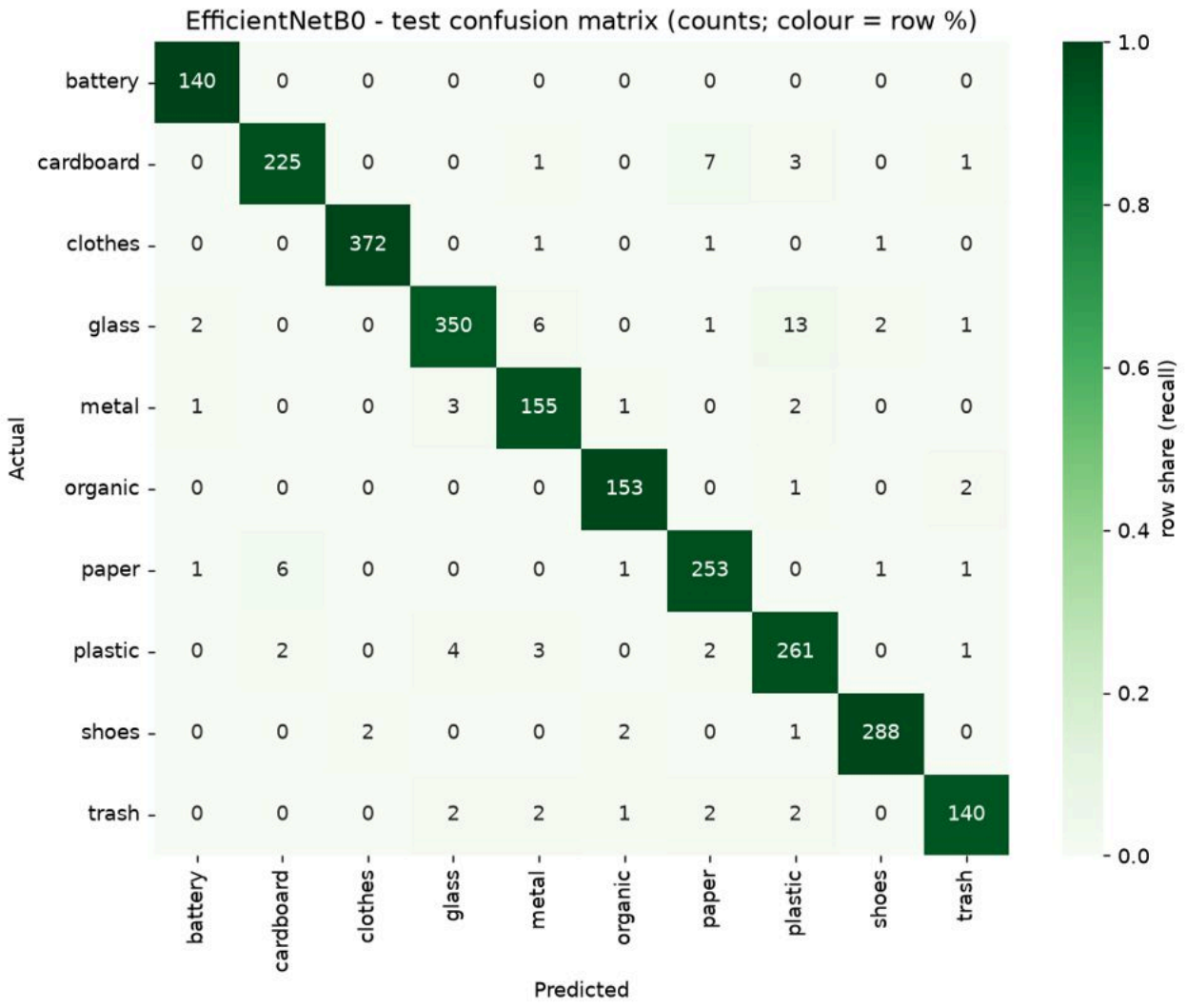



Validation accuracy and loss per epoch; the dotted line marks the start of fine-tuning.

## Statistical significance

McNemar's exact test compares the best model with each other model on the *same* test images (it counts the images only one of the two models gets right): MobileNetV2 $p < 0.001$; ResNet50 $p < 0.001$; Baseline CNN $p < 0.001$. A p-value below 0.05 means the difference in errors is unlikely to be chance.

## 6. Error analysis: the best model

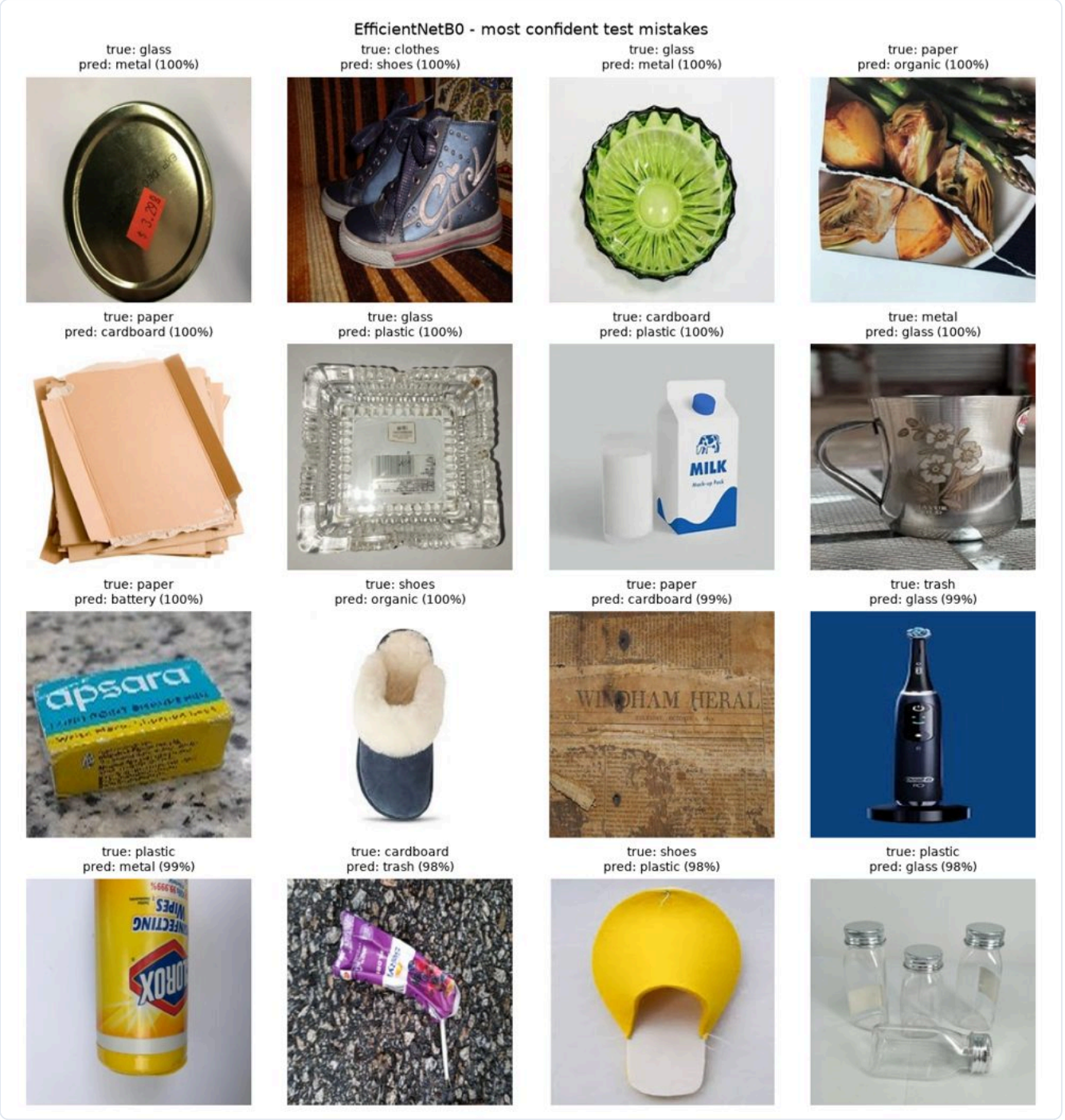



EfficientNetB0: test confusion matrix (numbers = counts; colour = share of the true class).

| Class | Precision | Recall | F1 | Test images |
|---|---|---|---|---|
| battery | 0.972 | 1.000 | 0.986 | 140 |
| cardboard | 0.966 | 0.949 | 0.957 | 237 |
| clothes | 0.995 | 0.992 | 0.993 | 375 |
| glass | 0.975 | 0.933 | 0.954 | 375 |
| metal | 0.923 | 0.957 | 0.939 | 162 |
| organic | 0.968 | 0.981 | 0.975 | 156 |
| paper | 0.951 | 0.962 | 0.957 | 263 |
| plastic | 0.922 | 0.956 | 0.939 | 273 |
| shoes | 0.986 | 0.983 | 0.985 | 293 |
| trash | 0.959 | 0.940 | 0.949 | 149 |

**Strongest classes:** clothes (F1 0.993), battery (F1 0.986), shoes (F1 0.985). **Weakest:** plastic (F1 0.939), metal (F1 0.939), trash (F1 0.949). **Most common confusions: glass → plastic** (13 images, 3% of glass), **cardboard → paper** (7 images, 3% of cardboard), **glass → metal** (6 images, 2% of glass), **paper → cardboard** (6 images, 2% of paper). These are physically plausible: clear plastic and glass bottles look alike, and paper and cardboard are the same material in different thicknesses. "Trash" is by definition a mixed, catch-all class.

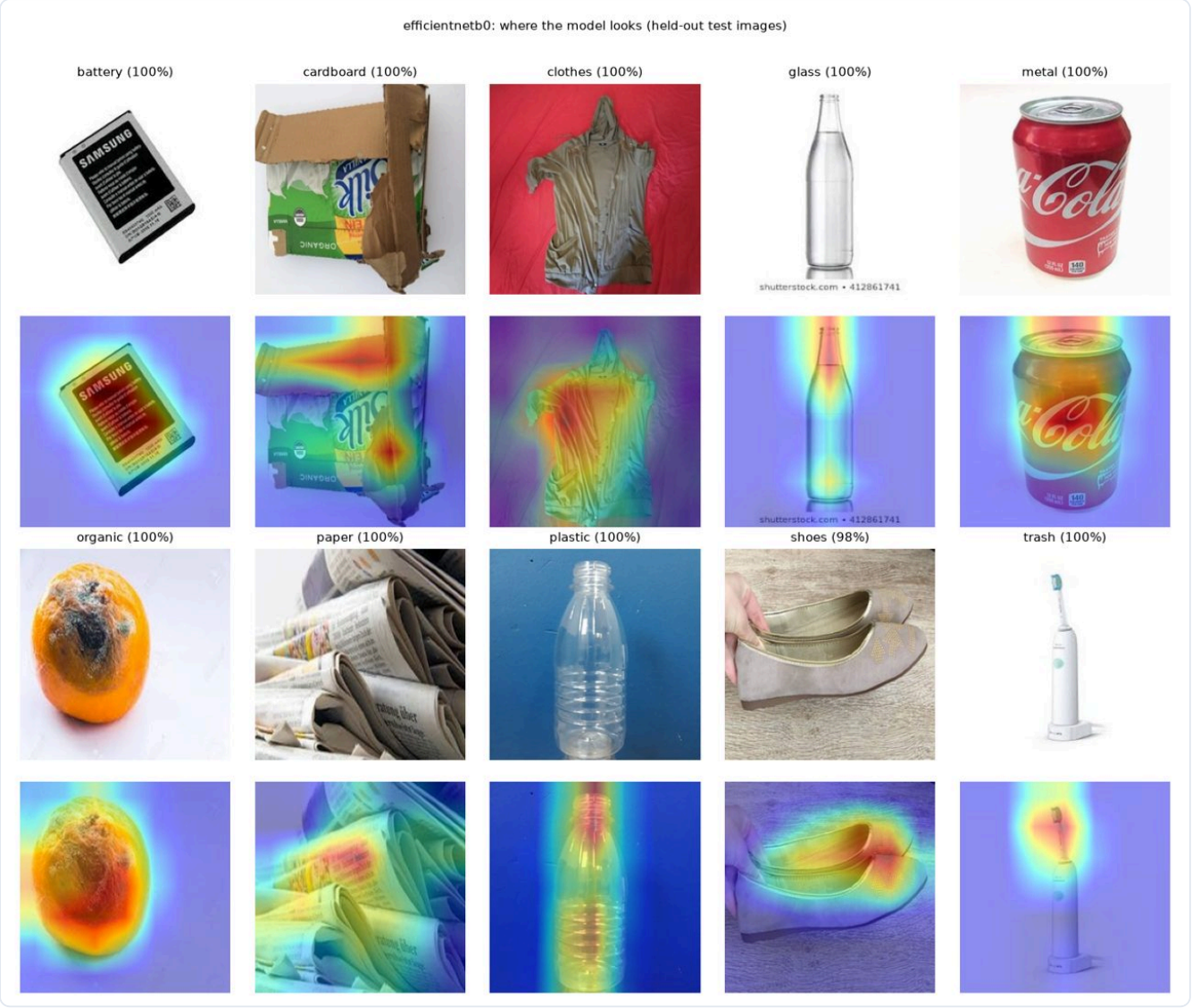
> **Some "mistakes" are label errors in the source data.** Among the most confident errors, an image labelled *glass* is clearly a metal lid (the model says metal), and one labelled *clothes* shows sneakers (the model says shoes). The true accuracy is therefore slightly *higher* than reported. Fixing such labels would be the next data-quality step, and the model's confident disagreements are a cheap way to find them.



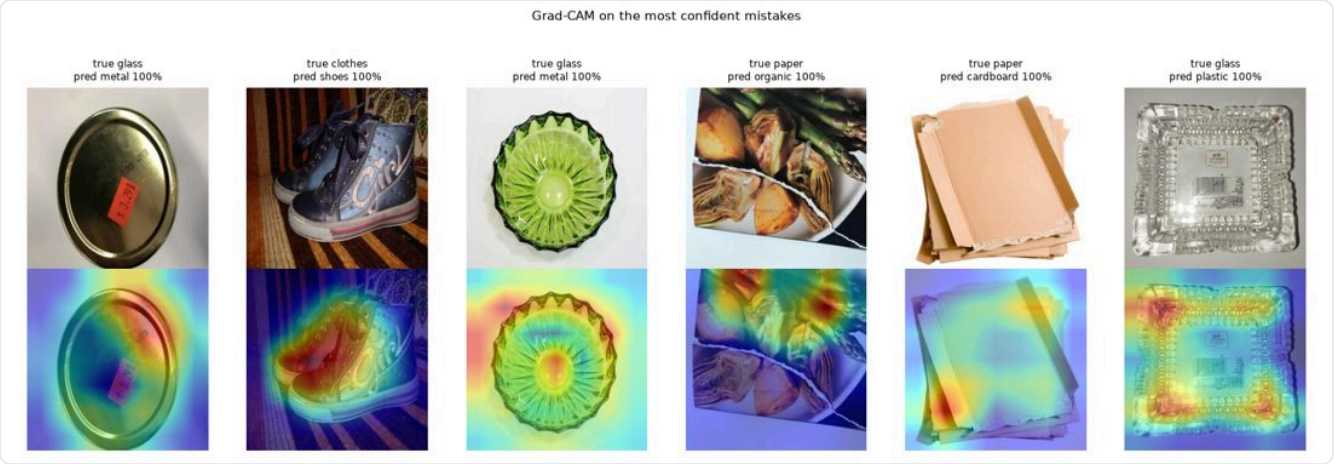

The most confident mistakes on the test set.

## 7. Explainability with Grad-CAM

Grad-CAM highlights the image regions that drove a prediction. It is computed from the class logit (pre-softmax), because at ~100% confidence the softmax saturates and its gradients vanish. On correct predictions the model focuses on the object itself, not the background. This matters because many training photos share white backgrounds, so a model could otherwise learn background shortcuts. The app shows this heatmap for every upload.




One held-out test image per class with its Grad-CAM heatmap.




Grad-CAM on the most confident mistakes. Several are mislabelled source images (a metal lid labelled glass, sneakers labelled clothes).

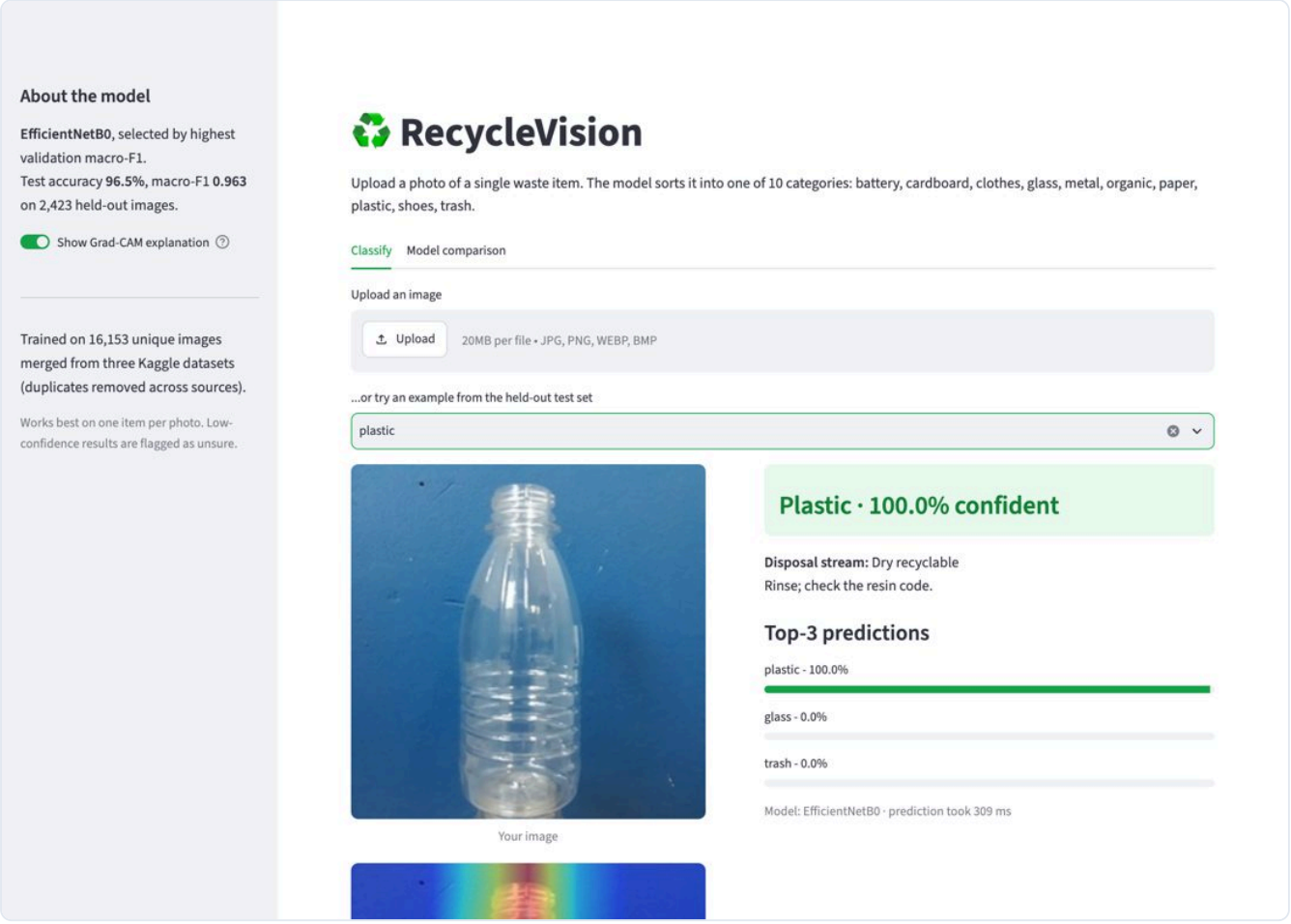
## 8. Efficiency and deployment

| Model | Parameters | File size | CPU compute (1 image) | Train time (T4) | Epochs | McNemar p vs best |
|---|---|---|---|---|---|---|
| **EfficientNetB0** | **4.4 M** | **18 MB** | **18 ms** | **15 min** | **23** | **-** |
| MobileNetV2 | 2.6 M | 11 MB | 12 ms | 11 min | 23 | < 0.001 |
| ResNet50 | 24.1 M | 97 MB | 38 ms | 15 min | 18 | < 0.001 |
| Baseline CNN | 1.2 M | 5 MB | 19 ms | 32 min | 25 | < 0.001 |

CPU compute is the model's time for one image on a laptop CPU (Apple M2), using the same compiled path as the app. In the running app one upload takes about 0.3 s end to end, because the first TensorFlow call of each Streamlit rerun carries a fixed overhead. Streamlit Cloud's shared CPU is slower, but a prediction still takes well under a second. The deployed model is 18 MB with 4.4 M parameters, well under GitHub's 100 MB file limit.

**App** ( `app/streamlit_app.py` ):

- Upload an image (or pick a held-out example), then get the predicted class with its confidence and the top-3 predictions.
- Disposal advice for the predicted category, plus a Grad-CAM overlay.
- Below 80% confidence the app says it is unsure instead of asserting a class. The threshold was chosen on the validation set: 5.2% of images are flagged, 61 of the 119 validation errors are among them, and accuracy on the remaining images is 97.5%.
- A model-comparison tab with the table and confusion matrix above.




The Streamlit app classifying a held-out test image.

## 9. Conclusions and recommendations

- **Transfer learning wins clearly:** every ImageNet backbone beats the from-scratch CNN by a wide margin, on 16,153 images.
- **Data hygiene mattered as much as the model:** 11,232 of 30,301 raw images were duplicates. Without de-duplication, copies of test photos would have been in the training set, so the test score would not measure generalisation.
- **Deployment choice:** EfficientNetB0 gives the best macro-F1. The table in Section 8 shows the speed and size trade-off if a smaller edge model is needed, e.g. for a bin.
- **Business suggestions:**
  - Use the confidence score to route low-confidence items to a human sorter (Section 8 shows the trade-off).
  - Collect real bin-camera images to fine-tune, since most training photos are clean single-object shots.
  - Batteries must never enter household recycling: the app already shows a hazardous-waste warning, and a smart bin should route them to a sealed compartment.
- **Limitations:**
  - One object per photo.
  - Web and studio images are cleaner than real waste streams.
  - "Trash" is heterogeneous.

## 10. Brief requirements checklist

| Requirement | Where |
|---|---|
| Dataset from the brief (Kaggle) | All three linked datasets, merged and de-duplicated (Section 2) |
| Cleaned & labelled dataset (deliverable) | `data/manifest.csv` + 224×224 image set (rebuilt by `scripts/build_dataset.py`; shared as a zip) |
| Resize 224×224, normalise, augment, handle imbalance | Sections 2 and 4; `data.py`, `models.py` |
| EDA: class counts, samples, colour distribution | Section 3; `scripts/run_eda.py` |
| Baseline CNN + transfer learning (MobileNetV2, EfficientNetB0, ResNet50) | Sections 4–5; `models.py` |
| Accuracy, precision, recall, F1, confusion matrix | Sections 5–6; `reports/metrics/` |
| Best model by F1 | Validation macro-F1 → EfficientNetB0 |
| Accuracy > 85%, balanced precision/recall, low false positives | 96.5%; P 0.962 / R 0.965; FPR 0.40% |
| Streamlit app: upload, label, confidence, top-3 | Section 8 (+ Grad-CAM and disposal advice) |
| Fast inference, reasonable model size | 18 ms model compute on CPU (about 0.3 s per upload in the app); 18 MB |
| Modular, documented code | `src/recyclevision/` package, scripts, README, tests |
| Deployment (Streamlit Cloud or local) | Runs locally; Streamlit Cloud steps in the README |
| Documentation: data handling, training, deployment | README, this report, module docstrings, presentation script |